

HBASE

Sure! Let's dive into the concept of HBase, which is a type of database.

HBase is like a giant digital filing cabinet that helps us store and manage large amounts of information in a way that is easy to access and update. Imagine you have a collection of books, and instead of organizing them by title or author, you group them by topics or themes. In HBase, we do something similar by organizing data into "column families." Each column family can hold different types of information, just like how a topic can include various books related to it. This makes it flexible and adaptable, allowing us to add new information without having to rearrange everything.

If you think of HBase as a library, the "HMaster" is like the librarian who manages the library, ensuring everything is in order and helping you find what you need. The "region servers" are like the shelves where the books (or data) are stored, and they help you retrieve the information quickly. This setup allows HBase to handle lots of data efficiently, making it perfect for applications that need to read and write data in real-time, like monitoring heart rates from hospital sensors.

HBase



Fault tolerance refers to the working ability of a system or computer to continue working even in unfavorable conditions such as when a server crashes



Column-oriented non-relational database management system



HBase runs on top of HDFS



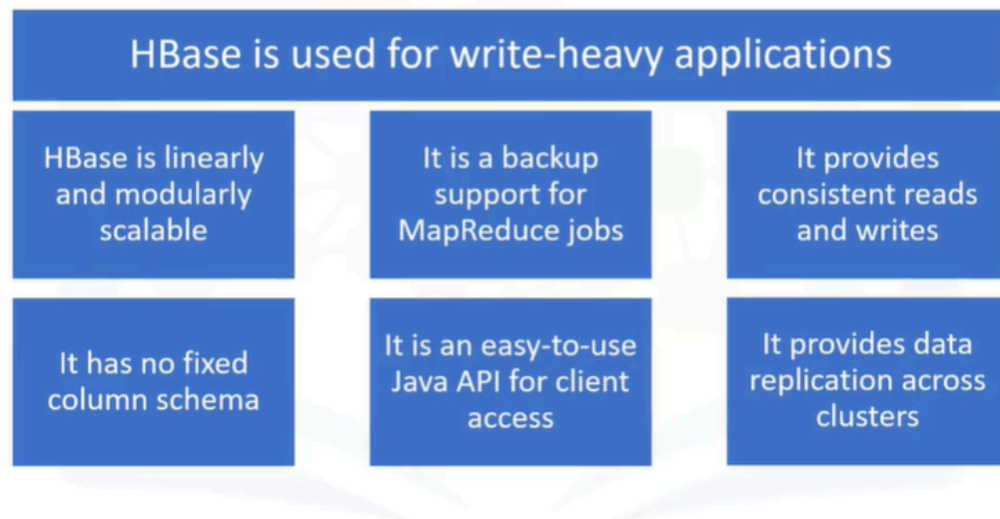
Provides a fault-tolerant way of storing sparse datasets



Works well with real-time data and random read and write access to Big Data

- HBase is a column-oriented non-relational database management system that runs on top of Hadoop Distributed File System, or HDFS. It provides a fault-tolerant way of storing sparse data sets. Fault tolerance refers to the working ability of a system or computer to continue working even in unfavorable conditions such as when a server crashes. It works well with real-time data and random read and write access to Big Data.

HBase features



- HBase is used for write-heavy applications, to store large amounts of datasets, and process and provide analytics in real time. It is referred to as linearly scalable because it supports scalability in both linear and modular form. HBase is a backup support for MapReduce jobs. HBase can be used for high-speed requirements because it offers consistent reads and writes. HBase has no fixed columns and is only defined by column families. It provides an easy-to-use Java API for client access. HBase supports data replication across clusters.

An example

Patient Details		Heart Rate	Time Stamp
Patient Name	Patient Age	Heart Rate	Time Stamp
Patient A	28	120 BPM	8:50 AM
Patient A	28	110 BPM	10:10 AM
Patient B	77	95 BPM	11:00 AM
Patient C	45	150 BPM	11:30 AM
:			
Patient B	77	115 BPM	12:30 AM

A few things to note:

- Predefine the table schema and specify column families
- New columns can be added to column families at anytime
- HBase schema is very flexible
- HBase has master nodes to manage the cluster and region servers to perform the work

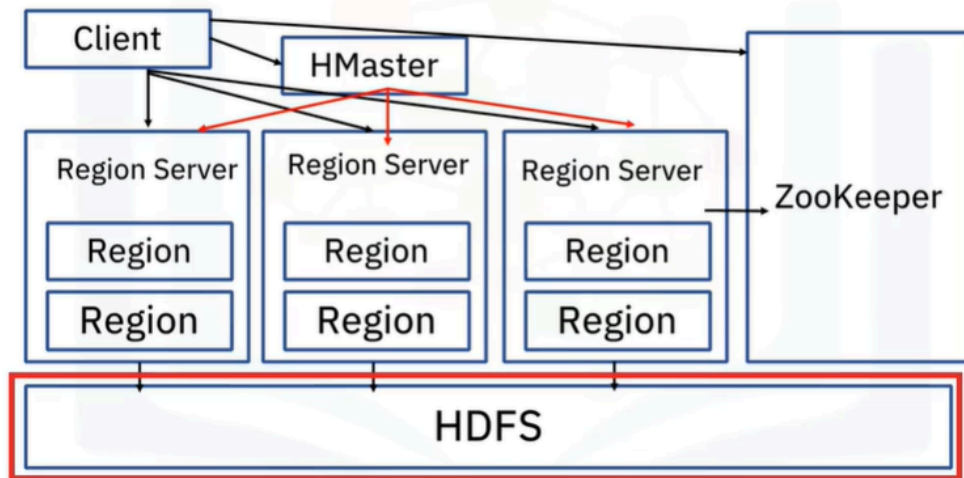
- An HBase column represents an attribute of an object.
- Let's look at an example. If the table is storing sensors from heart monitors in the hospital, each row might be a log record. A typical column could contain information about the patient's details or the time when the log record was taken. HBase allows for many attributes to be grouped together into column families such that the elements of a column family are all stored together. For example, "Patient Name" and "Patient Age" will be stored as "Patient Details." The columns in HBase will look like – Patient Details, Heart Rate, and Time Stamp. When creating schemas in HBase, you must predefine the table schema and specify the column families. New columns can be added to families at any time, making the schema flexible and adaptable to changing application requirements. Just as HDFS has a Name Node and secondary nodes, HBase is built on similar concepts. In HBase, a primary node manages the cluster and a region server stores portions of the tables and performs the work on the data.

Differences between HBase and HDFS

HBase	HDFS
HBase stores data in the form of columns and rows in a table	HDFS stores data in a distributed manner across different nodes on that network
HBase allows dynamic changes	HDFS has a rigid architecture that doesn't allow changes
HBase is suitable for random writes and reads of data stored in HDFS	HDFS is suited for write once and read many times
HBase allows for storing and processing of Big Data	HDFS is for storing only

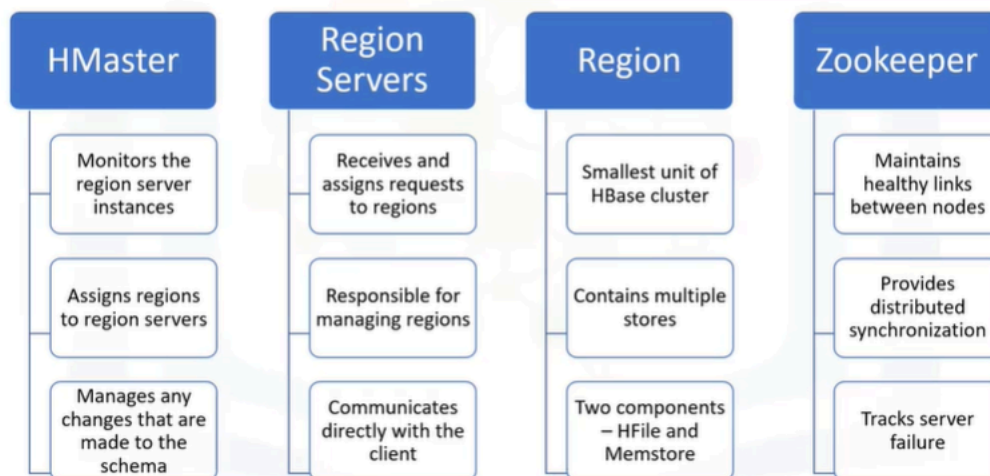
- While HBase and HDFS are both used to store massive amounts of data, one of the major differences is that HBase stores data in the form of columns and rows in a table while HDFS stores it in a distributed manner across different nodes on that network.
- HBase allows dynamic changes while HDFS has a rigid architecture that doesn't allow changes. HBase is suitable for read, write, update, and delete type workloads on a record level. HDFS is just an underlying level resembling properties of a file system, so is more suited for write once and read in batch workloads.
- HBase allows for storing and processing of Big Data while HDFS is for storing only.

HBase architecture



- The HBase architecture consists mainly of four components: 1. HMaster 2. HRegionserver 3. HRegions 4. ZooKeeper From the architecture, we can see that HBase sits and works on top of HDFS. HDFS provides a distributed environment for the storage, and it is a file system designed to run on commodity hardware. It stores each file in multiple blocks, and to maintain fault tolerance, it replicates the blocks across a Hadoop cluster.

HBase concepts



- HMaster is the master server. It monitors the region server instances, assigns regions to region servers, and distributes services to different region servers, and manages any changes that are made to the schema and metadata operations. The region servers receive read and write requests from the client and assign the request to a specific region where the column family resides. They are responsible for serving and managing regions that are present in a distributed cluster. Rather than go through HMaster all the time, the region servers can communicate directly with the client to facilitate requests. The region is the basic building element and smallest unit of the HBase cluster that consist of column families. It contains multiple stores, one for each column family and has two components – HFile and Memstore. ZooKeeper is a centralized service for maintaining configuration information to maintain healthy links between nodes. It provides synchronization across distributed applications and is used to track server failure and network partitions by triggering an error message, and then starts repairing the failed nodes.

Summary

In this video, you learned that:

- HBase is a column-oriented non-relational database management system that runs on HDFS
- HBase columns are defined by column families
- HBase is linearly scalable, highly efficient and provides an easy-to-use Java API for client access
- A key difference between HDFS and HBase is that HBase allows dynamic changes compared to the rigid architecture of HDFS
- HBase architecture consists of HMaster, region servers, region, Zookeeper and HDFS